Property Maintenance and Service Coordination System

Database Schema Design

The following SQL script defines the relational database structure for the **Community Maintenance System** based on the provided Entity-Relationship (ER) diagram. This schema is designed to manage users, maintenance requests, service provider assignments, billing, and communication within a residential community.

The architecture utilizes a **Generalization/Specialization (Is-A)** hierarchy, where Users serves as the parent entity for Owner, Maintenance_Manager, and Service_provider. This ensures data integrity by centralizing core profile information (name, email, credentials) while allowing role-specific attributes to be stored in separate tables.

1. Strong Entity

A **Strong Entity** is an independent entity that contains its own unique Primary Key. It does not require any other entity to exist in the database.

Relational Schemas (Strong Entities):

1. **Administrator** (admin_id [PK], name, email, community_name, password)
2. **Users** (user_id [PK], name, email, phone, password, *admin_id* [FK])
3. **Maintenance_Manager** (manager_id [PK/FK], community_name)
4. **Owner** (owner_id [PK/FK], community_name, property_unit)
5. **Service_Provider** (provider_id [PK/FK], category)
6. **Complaint** (complaint_id [PK], *owner_id* [FK], *manager_id* [FK], category, description, photo, status, date)
7. **Service_Assignment** (assignment_id [PK], *complaint_id* [FK], *provider_id* [FK], *manager_id* [FK], due_date)
8. **Service_Bill** (bill_id [PK], *complaint_id* [FK], amount, penalty, date)
9. **Payment** (payment_id [PK], *bill_id* [FK], amount, payment_date)
10. **Maintenance_Cost_Payment** (maintenance_id [PK], *owner_id* [FK], *manager_id* [FK], month, maintenance_amount)
11. **Notification** (notification_id [PK], *user_id* [FK], *complaint_id* [FK], message, type, status, date_time)

2. Weak Entity

A **Weak Entity** is an entity that cannot be uniquely identified by its own attributes alone. It depends on a **Strong Entity** for its existence and identification.

Relational Schemas (Weak Entities):

1. **Rating** (owner_id [PK/FK], complaint_id [PK/FK], score, feedback, date)
   Dependency: Depends on both the Owner and the Complaint to exist.
2. **Service_Estimate** (assignment_id [PK/FK], estimated_cost, date)
   Dependency: Depends entirely on the Service_Assignment.

3.Normalization

1NF (First Normal Form):

Normalization to 1NF reduced repeating attributes and multi-valued fields in the relations.

• Multiple service provider details within a single complaint record are separated into the SERVICE_ASSIGNMENT entity instead of storing multiple times in COMPLAINT entity.

• Service estimation details were separated from the service assignment to avoid storing repeatedly within the same table.

• All attributes in all entities are ensured to be atomic.

• Multiple payments and billing details were separated into SERVICE_BILL and PAYMENT relations instead of directly inside the complaint or service assignment tables.

2NF (Second Normal Form):

Normalization to 2NF reduced partial dependencies on composite keys.

• Service estimation details were separated from assignment details:

SERVICE_ESTIMATE(assignment_id, estimated_cost, date)

• Maintenance cost payments were separated from owner records:

MAINTENANCE_COST_PAYMENT(maintenance_id, owner_id, manager_id, month, maintenance_amount)

3NF (Third Normal Form):

Normalization to 3NF reduced transitive dependencies between non-key attributes.

• Owner, Administrator and Maintenance Manager information was separated from

complaint records:

OWNER(owner_id, name, contact, email, property_unit, community_name, password)

ADMINISTRATOR(admin_id, name, email, community_name, password)

MAINTENANCE_MANAGER(manager_id, name, email, phone, community_name, password)

• Service billing and payment information were separated:

SERVICE_BILL(bill_id, complaint_id, amount, penalty, date)

PAYMENT(payment_id, bill_id, amount, payment_date)

• Service ratings and feedback were separated from complaint records:

RATING(owner_id, complaint_id, score, feedback, date)

BCNF (Boyce–Codd Normal Form):

Normalization to BCNF ensured that every relation is fully dependent on its primary key, further

reducing anomalies.

• Each entity has a clear primary key dependency:

OWNER → owner_id

MAINTENANCE_MANAGER → manager_id

SERVICE_PROVIDER → provider_id

ADMINISTRATOR → admin_id

COMPLAINT → complaint_id

SERVICE_ASSIGNMENT → assignment_id

SERVICE_BILL → bill_id

PAYMENT → payment_id

• Attributes such as complaint_id, assignment_id, bill_id, and payment_id uniquely determine all other attributes in their relations.

• Relations like SERVICE_ESTIMATE, SERVICE_ASSIGNMENT, SERVICE_BILL, and PAYMENT store only attributes directly related to their primary keys.

4.Cardinality

Administrator to Users (1:m): One Admin manages many Users.

Users to Notification (1:m): One User receives many Notifications.

Complaint to Notification (1:m): One specific Complaint can generate multiple status update Notifications.

Owner to Complaint (1:m): One Owner can raise many Complaints.

Maintenance_Manager to Complaint (1:m): One Manager oversees many Complaints.

Maintenance_Manager to Service_Assignment (1:m): One Manager assigns many tasks.

Service_Provider to Service_Assignment (1:m): One Provider is given many Assignments.

Complaint to Service_Assignment (1:m): One Complaint links to many Assignments.

Service_Assignment to Service_Estimate (1:1): One Assignment has one specific cost Estimate.

Complaint to Service_Bill (1:1): Each Complaint leads to one unique Bill.

Service_Bill to Payment (1:1): One Bill is settled by one specific Payment.

Owner to Rating (1:m): One Owner gives many Ratings (across different complaints).

Complaint to Rating (1:1): Each specific Complaint receives only one final Rating.

Owner to Maintenance_Cost_Payment (1:m): One Owner makes many monthly dues payments.

Maintenance_Manager to Maintenance_Cost_Payment (1:m): One Manager logs many different payments.

```
CREATE DATABASE PropSync;
USE PropSync;
```

1.Administrator

```
CREATE TABLE Administrator (     admin_id INT
AUTO_INCREMENT PRIMARY KEY,     name
VARCHAR(100) NOT NULL,     email VARCHAR(150)
UNIQUE NOT NULL,     community_name
VARCHAR(150),     password VARCHAR(255) NOT
NULL
);
```

2.Users

```
CREATE TABLE Users (     user_id INT
AUTO_INCREMENT PRIMARY KEY,     name
VARCHAR(100) NOT NULL,     email
VARCHAR(150) UNIQUE NOT NULL,     phone
VARCHAR(20),     password VARCHAR(255) NOT
NULL,     admin_id INT,
   CONSTRAINT fk_users_admin FOREIGN KEY (admin_id) REFERENCES
Administrator(admin_id)
);
```

3.Maintenance_Manager

```
CREATE TABLE Maintenance_Manager (
manager_id INT PRIMARY KEY,     community_name
VARCHAR(150),
   CONSTRAINT fk_manager_user FOREIGN KEY (manager_id) REFERENCES
Users(user_id)
);
```

4.Owner

```
CREATE TABLE Owner (     owner_id
INT PRIMARY KEY,
community_name VARCHAR(150),
   property_unit VARCHAR(50) NOT NULL,
   CONSTRAINT fk_owner_user FOREIGN KEY (owner_id) REFERENCES Users(user_id)
);
```

5.Service_Provider

```
CREATE TABLE Service_Provider (
provider_id INT PRIMARY KEY,     category
VARCHAR(100) NOT NULL,
   CONSTRAINT fk_provider_user FOREIGN KEY (provider_id) REFERENCES Users(user_id)
);
```

6.Complaint

```
CREATE TABLE Complaint (     complaint_id INT
AUTO_INCREMENT PRIMARY KEY,     owner_id INT
NOT NULL,     manager_id INT NOT NULL,     category
VARCHAR(100),     description VARCHAR(500),     photo
VARCHAR(255),
   status VARCHAR(50) DEFAULT 'Pending',
date DATE NOT NULL,
   CONSTRAINT fk_complaint_owner FOREIGN KEY (owner_id) REFERENCES
Owner(owner_id),
   CONSTRAINT fk_complaint_manager FOREIGN KEY (manager_id) REFERENCES
Maintenance_Manager(manager_id)
);
```

7.Service_Assignment

```
CREATE TABLE Service_Assignment (     assignment_id
INT AUTO_INCREMENT PRIMARY KEY,     complaint_id
INT NOT NULL,     provider_id INT NOT NULL,
manager_id INT NOT NULL,     due_date DATE,
   CONSTRAINT fk_assign_complaint FOREIGN KEY (complaint_id) REFERENCES
Complaint(complaint_id),
   CONSTRAINT fk_assign_provider FOREIGN KEY (provider_id) REFERENCES
Service_Provider(provider_id),
   CONSTRAINT fk_assign_manager FOREIGN KEY (manager_id) REFERENCES
Maintenance_Manager(manager_id)
);
```

8.Service_Estimate

```
CREATE TABLE Service_Estimate ( assignment_id
   INT PRIMARY KEY, estimated_cost
   DECIMAL(10, 2) NOT NULL,
   date DATE,
   CONSTRAINT fk_estimate_assignment FOREIGN KEY (assignment_id) REFERENCES
Service_Assignment(assignment_id)
);
```

9.Rating

```
CREATE TABLE Rating (     owner_id INT NOT
NULL,     complaint_id INT NOT NULL,     score
INT CHECK (score BETWEEN 1 AND 5),
feedback VARCHAR(500),     date DATE,
   PRIMARY KEY (owner_id, complaint_id),
   CONSTRAINT fk_rating_owner FOREIGN KEY (owner_id) REFERENCES Owner(owner_id),
   CONSTRAINT fk_rating_complaint FOREIGN KEY (complaint_id) REFERENCES
Complaint(complaint_id)
);
```

10.Service_Bill

```
CREATE TABLE Service_Bill (     bill_id INT
AUTO_INCREMENT PRIMARY KEY,
complaint_id INT NOT NULL,     amount
DECIMAL(10, 2) NOT NULL,     penalty
DECIMAL(10, 2) DEFAULT 0.00,     date DATE,
   CONSTRAINT fk_bill_complaint FOREIGN KEY (complaint_id) REFERENCES
Complaint(complaint_id)
);
```

11.Payment

```
CREATE TABLE Payment (     payment_id INT
AUTO_INCREMENT PRIMARY KEY,     bill_id INT NOT
NULL,     amount DECIMAL(10, 2) NOT NULL,
payment_date DATE,
   CONSTRAINT fk_payment_bill FOREIGN KEY (bill_id) REFERENCES Service_Bill(bill_id)
);
```

12.Maintenance_Cost_Payment

```
CREATE TABLE Maintenance_Cost_Payment (
maintenance_id INT AUTO_INCREMENT PRIMARY KEY,
owner_id INT NOT NULL,
   manager_id INT NOT NULL,
   month VARCHAR(20),
   maintenance_amount DECIMAL(10, 2) NOT NULL,
   CONSTRAINT fk_mcp_owner FOREIGN KEY (owner_id) REFERENCES Owner(owner_id),
   CONSTRAINT fk_mcp_manager FOREIGN KEY (manager_id) REFERENCES
Maintenance_Manager(manager_id)
);
```

13.Notification

```
CREATE TABLE Notification (     notification_id INT
AUTO_INCREMENT PRIMARY KEY,     user_id INT NOT
NULL,     complaint_id INT,
   message VARCHAR(500) NOT NULL,
type VARCHAR(50),     status
VARCHAR(20),
   date_time DATETIME DEFAULT CURRENT_TIMESTAMP,
   CONSTRAINT fk_notif_user FOREIGN KEY (user_id) REFERENCES Users(user_id),
   CONSTRAINT fk_notif_complaint FOREIGN KEY (complaint_id) REFERENCES
Complaint(complaint_id)
);
```